Deep learning: processing sequence data

Processing sequence data

One area of success of DL is the ability to process sequential data, in particular textual and audio, as well as time series, with good performance

One of the most commonly used topologies in these applications are recurrent neural networks with various configurations

Applications include:

- NLP tasks (see next slide)
- Processing of audio and musical signals
- Time series forecasting
- Prediction of protein function, structure, and properties (in Bioinformatics)
- QSAR/ predicting activities of chemical compounds

Some tasks in Natural Language Processing (NLP)

- “What’s the topic of this text?” - **text classification**
- “Does this text contain abuse?” - **content filtering**
- “Does this text sound positive or negative?” - **sentiment analysis**
- “What should be the next word in this incomplete sentence?” - **language modeling**
- “How would you say this in German?” - **translation**
- “How would you summarize this article in one paragraph?” - **summarization**

Processing texts

DL models only process numerical data, and thus do not accept raw text – need to pre-process texts – ***vectorization*** – to convert into numerical vectors

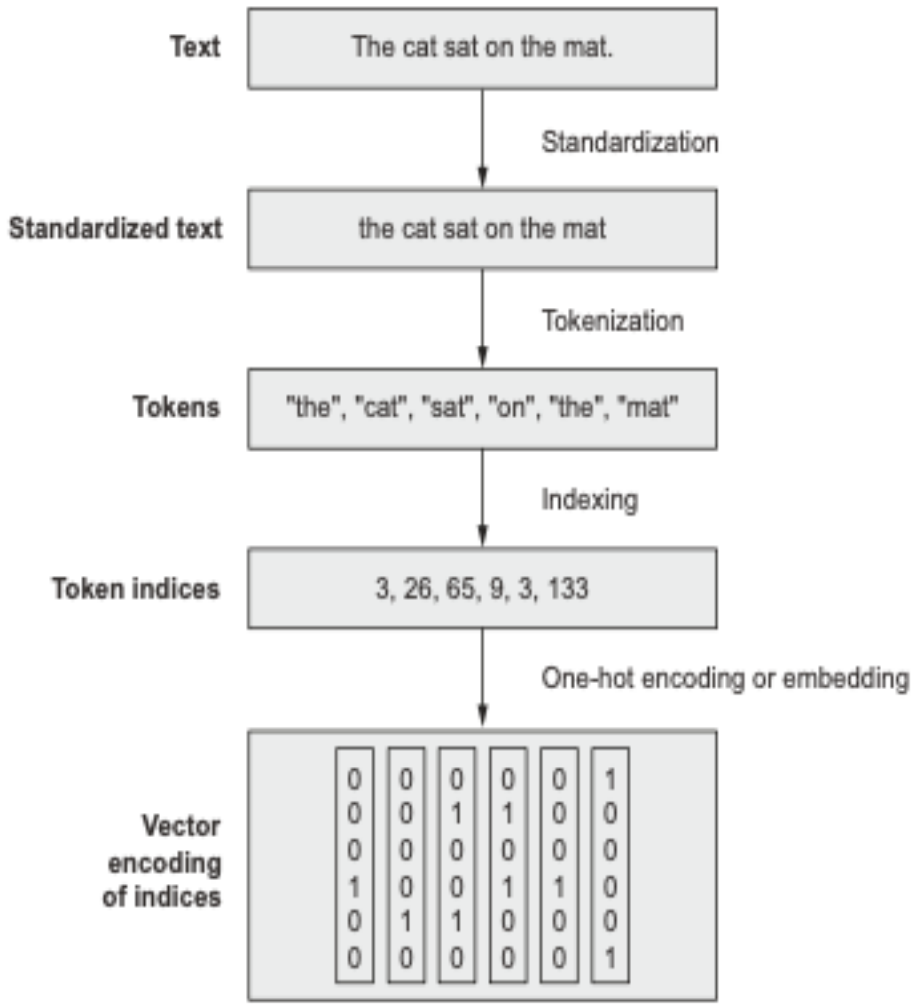
Several steps are usually taken:

- ***Standardization*** – removing punctuation; upper case to lower case; stemming; etc
- ***Tokenization*** - splitting the text into units (tokens), that may be words, groups of words, characters, groups of characters
- ***Indexing*** the tokens – converting into a numerical vector (e.g. one-hot encoding)

Processing texts

Typically, we see texts as word sequences

Texts are converted to numeric vectors, where words (or other types of tokens) are encoded as indexes of a vocabulary that typically has a predefined maximum size



Tokenization

- **Word-level tokenization**— tokens are space-separated (or punctuation-separated) substrings. A variant is to split words into subwords when applicable—for instance, treating “staring” as “star+ing” or “called” as “call+ed.”
- **N-gram tokenization**— tokens are groups of N consecutive words. For instance, “the cat” or “he was” would be 2-gram tokens (also called bigrams).
- **Character-level tokenization**— each character is its own token. In practice, this scheme is rarely used, only in specialized contexts, like text generation or speech recognition.

```
def tokenize(text):
    text = text.lower()
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"[^a-zA-Z]", " ", text)
    return text.split()
```

Simple example of a word-level tokenizer with previous standardization

Jupyter Notebook:
examples-text-processing.ipynb

Encoding: example

```
def build_vocab(texts, max_words=100):
    counter = Counter()
    for text in texts:
        counter.update(tokenize(text))
    most_common = counter.most_common(max_words)

    vocab = {"<pad>": 0, "<unk>": 1}
    for i, (word, _) in enumerate(most_common, start=2):
        vocab[word] = i
    return vocab
```

Example of a simple function to build a vocabulary

```
def encode(vocab, text, max_len=100):
    tokens = tokenize(text)
    ids = [vocab.get(token, vocab["<unk>"])
       for token in tokens]
    return ids[:max_len]
```

Simple example of an encoder of a text
(to integer indexes)

The set of available texts (training sets) are organized into a **vocabulary** of fixed size. Usually, a dictionary with words as keys and integer indexes as values.
Only the **`max_words`** most common words are kept. Usually, index 0 is for "mask" used for instance in padding
Index 1 is used for words not in vocabulary

Jupyter Notebook:
examples-text-processing.ipynb

Encoding: example

```
def one_hot_encode(vocab, token):
    vector = np.zeros((len(vocab),))
    token_index = vocab[token]
    vector[token_index] = 1
    return vector
```

Simple example of an one hot encoder

```
> vocab = build_vocab(["the cat sat on the mat", "the cat is fat"])
> vocab
{'<pad>': 0, '<unk>': 1, 'the': 2, 'cat': 3, 'sat': 4, 'on': 5, 'mat': 6, 'is': 7, 'fat': 8}
> encode(vocab, "the mat is a cat")
[2, 6, 7, 1, 3]
> one_hot_encode(vocab, "mat")
array([0., 0., 0., 0., 0., 0., 1., 0., 0.])
```

One-hot encoding each word (token) has an index; representation of a word (token) is a vector with a value of 1 in that index and zeros in the remaining

Jupyter Notebook:
examples-text-processing.ipynb

One-hot encoding: example

V = [a, aaron, …, zulu, <UNK>] **Vocabulary**

Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)

$$
\begin{array}{c}5391\end{array}\begin{bmatrix}0\\0\\0\\0\\\vdots\\1\\\vdots\\0\\0\end{bmatrix}
\quad
\begin{array}{c}9853\end{array}\begin{bmatrix}0\\0\\0\\0\\0\\\vdots\\1\\\vdots\\0\end{bmatrix}
\quad
\begin{array}{c}4914\end{array}\begin{bmatrix}0\\0\\0\\\vdots\\1\\\vdots\\0\\0\\0\end{bmatrix}
\quad
\begin{array}{c}7157\end{array}\begin{bmatrix}0\\0\\0\\0\\0\\\vdots\\1\\\vdots\\0\end{bmatrix}
\quad
\begin{array}{c}456\end{array}\begin{bmatrix}0\\\vdots\\1\\\vdots\\0\\0\\0\\0\\0\end{bmatrix}
\quad
\begin{array}{c}6257\end{array}\begin{bmatrix}0\\0\\0\\0\\0\\\vdots\\1\\\vdots\\0\end{bmatrix}
$$

$|V| = 10000$

Andrew Ng

Representing groups of words in ML

- Counts of (or presence) of words or tokens (also known as **bags of words**) can can be used as features in Machine Learning approaches (tabular datasets)
- In this case, the order of the words is neglected, so we treat each example as a **set** of words or n-grams
- Main advantage: represent each example text as a vector, where each entry is a presence indicator/ count for a given word/ n-gram. The vector has as many dimensions as words in the vocabulary
- Main disadvantage: we do not consider the order of the words

Representing order of words in ML

- By considering N-grams we can recover local order information
- To avoid combinatorial explosion, **bigrams** (pairs of words/ tokens) are the mostly used
- Therefore, the presence or counts of bigrams will be the considered features in this approach, still keeping a tabular dataset but increasing significantly the number of features
- As before we can use presence/ absence (binary) or counts

TF-IDF normalization

- Objective of normalization is to take counts of words or N-grams considering the number of times a word typically occurs in the language
- TF-IDF “term frequency, inverse document frequency” is the most used method for normalization
- It divides term frequency (how many times the term appears in the current document) by “document frequency” which estimates how often the term comes up across the dataset
- So, a term will be important if it occurs much more frequently in the given document than in the overall dataset
- TF-IDF may be used to create normalized features from word/N-gram counts

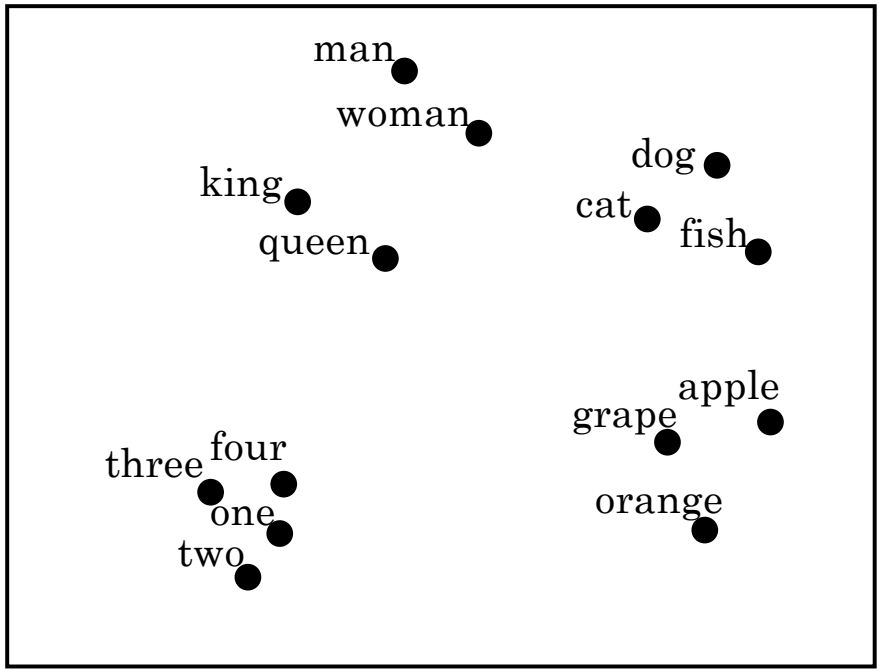
Word embeddings

Each word is represented by a relatively small "dense" numeric vector (e.g. size 256-1024)

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1.0	1.0	-0.95	0.97	0.00	0.01
Royal	0.02	0.01	0.93	0.95	-0.01	0.00
Age	0.41	0.38	0.7	0.69	0.03	-0.02
Food	0.01	0.01	0.02	0.01	0.95	0.97

Andrew Ng

Visualizing word embeddings/ analogies



$$e_{man} - e_{woman} \approx e_{king} - e_?$$

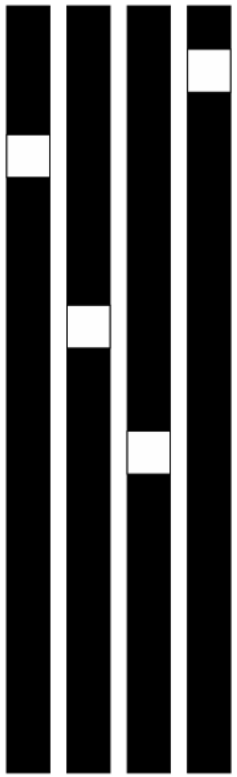
MAX:

$$sim(e_w, e_{king} - e_{man} + e_{woman})$$

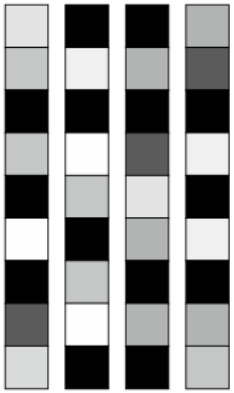
t-SNE Allows to represent 2D embeddings keeping distances

Andrew Ng

Word embeddings: characteristics



One-hot encoding



Embeddings

Word embeddings are

- Dense representations (vs sparse representations in one-hot encoding)
- Lower dimension
- Continuous values
- **Learned from data**

Learning embeddings

Embedding – matrix **E**:

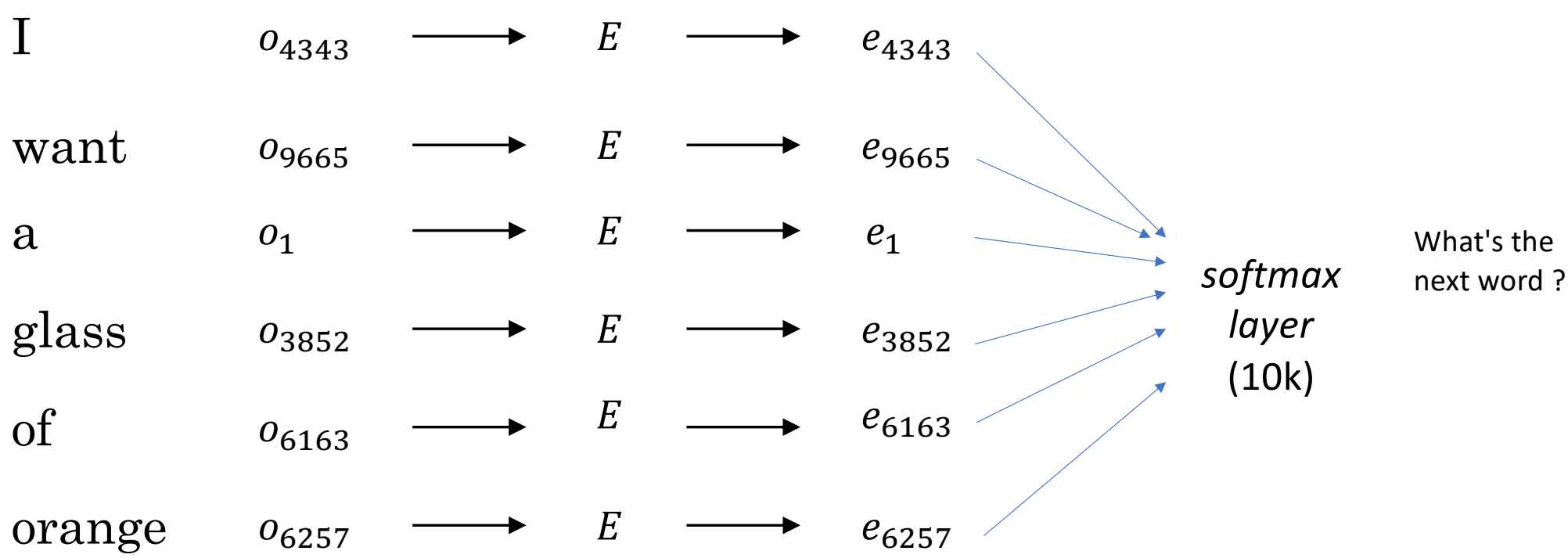
- rows: dimensions of the embedding
- columns: words of the vocabulary

Note that by multiplying the representation of a word as *one-hot encoding* by E we have the column that corresponds to the embedding vector of that same word

Values of this matrix E can be learned as weights in any neural network by gradient descent, being matrix E the first layer, having as inputs one or more words and can have any type of task in subsequent layers

Learning embeddings: example

I	want	a	glass	of	orange	_____.
4343	9665	1	3852	6163	6257	



[Bengio et. al., 2003, A neural probabilistic language model]

Example Embedding - IMDB

```
class EmbeddingClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.fc = nn.Linear(embed_dim, 1)
        print(self.embedding.weight.numel())

    def forward(self, x):
        # x shape: (batch_size, max_len)
        embedded = self.embedding(x)
        # shape: (batch_size, max_len, embed_dim)

        pooled = embedded.mean(dim=1)
        # shape: (batch_size, embed_dim)

        output = self.fc(pooled)
        return output.squeeze(1)
```

imdb_torch_full.py

Embeddings*:* transfer learning

Embeddings can be learned from scratch for each problem (if there is enough data)

In many cases, it is preferable to re-use pre-trained embeddings in extended datasets with large vocabularies – *transfer learning*

If we have enough data, we can always refine embeddings with the available data

Embeddings most used: Word2Vec, Glove

Example with Glove for the iMDB dataset:
Load Glove from https://huggingface.co/datasets/SLU-CSCI4750/glove.6B.100d.txt/tree/main

Recurrent neural networks

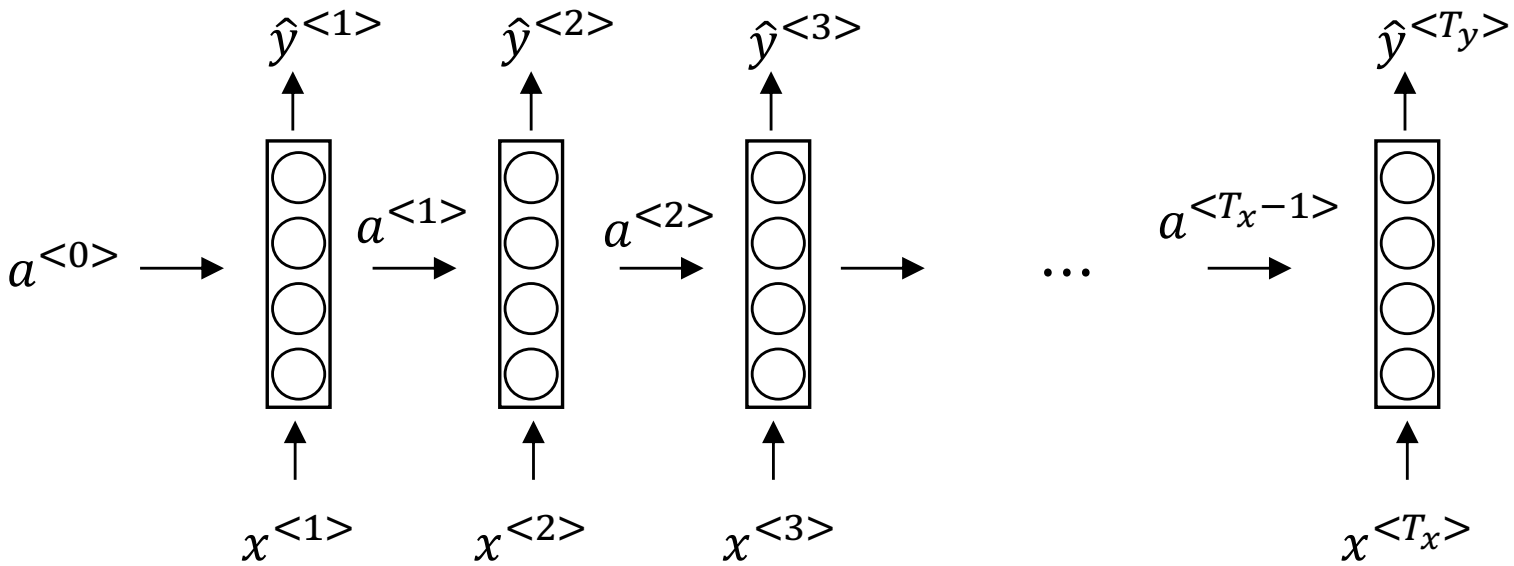
Unlike feedforward networks, recurrent networks have a topology that can contain cycles, with feedback from network outputs to inputs

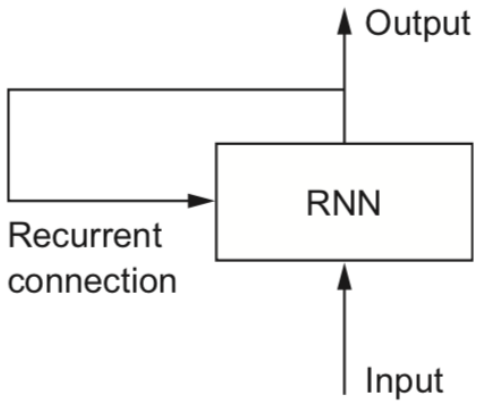
Thus, they have the ability to process sequential data, and can maintain and can have various types of state / "memory" (of "short" and "long" term)

For example, in the case of IMDB, each review can be processed sequentially by being maintained a state relative to the words already seen (even if each review is analyzed independently)

Training algorithms based on gradient descent (e.g Backprop through time) propagate errors through the chain rule from the last state to the initial state

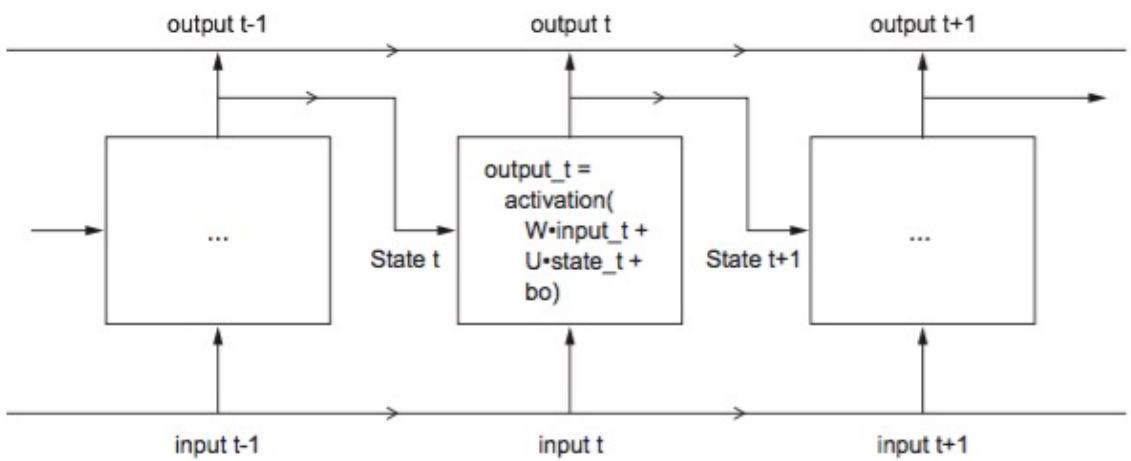
Recurrent neural networks





Calculation of outputs and states →

← Propagation of error



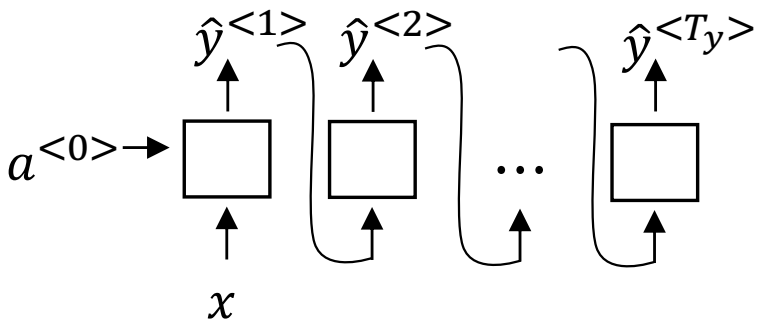
RNN – simple implementation

```
import numpy as np

timesteps = 100
input_features = 32
output_features = 64
inputs = np.random.random((timesteps, input_features))
state_t = np.zeros((output_features,))
W = np.random.random((output_features, input_features))
U = np.random.random((output_features, output_features))
b = np.random.random((output_features,))

successive_outputs = []
for input_t in inputs:
        output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)
        successive_outputs.append(output_t)
        state_t = output_t
final_output_sequence = np.stack(successive_outputs, axis=0)
```

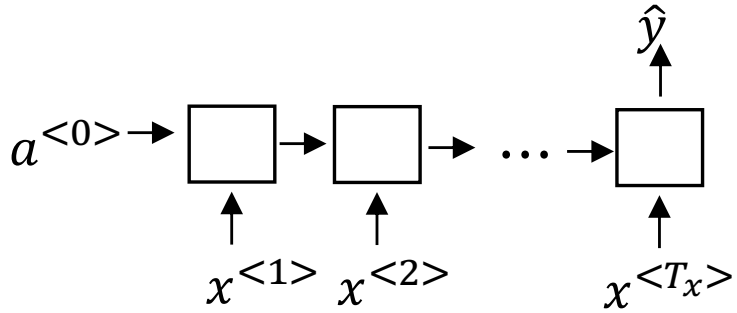
This part would need to be replaced by the real inputs

For a real example, weights would be trained !

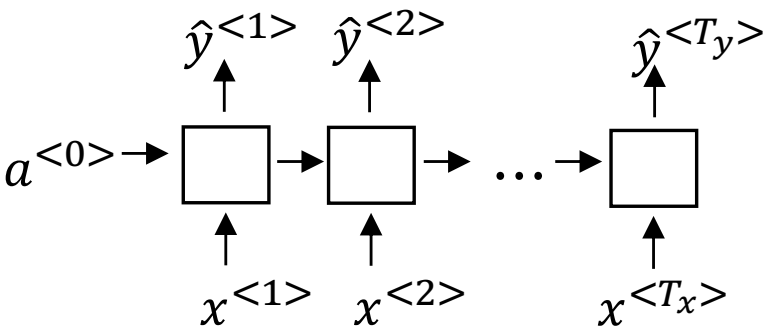
Several types of RNNs



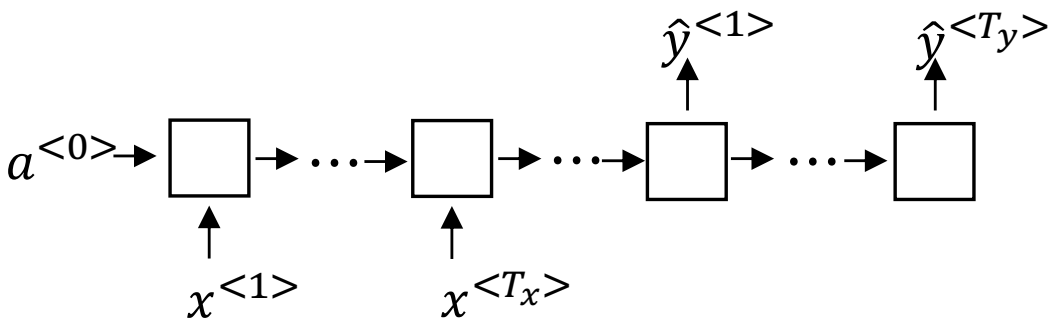
One to many



Many to one



Many to many



Many to many

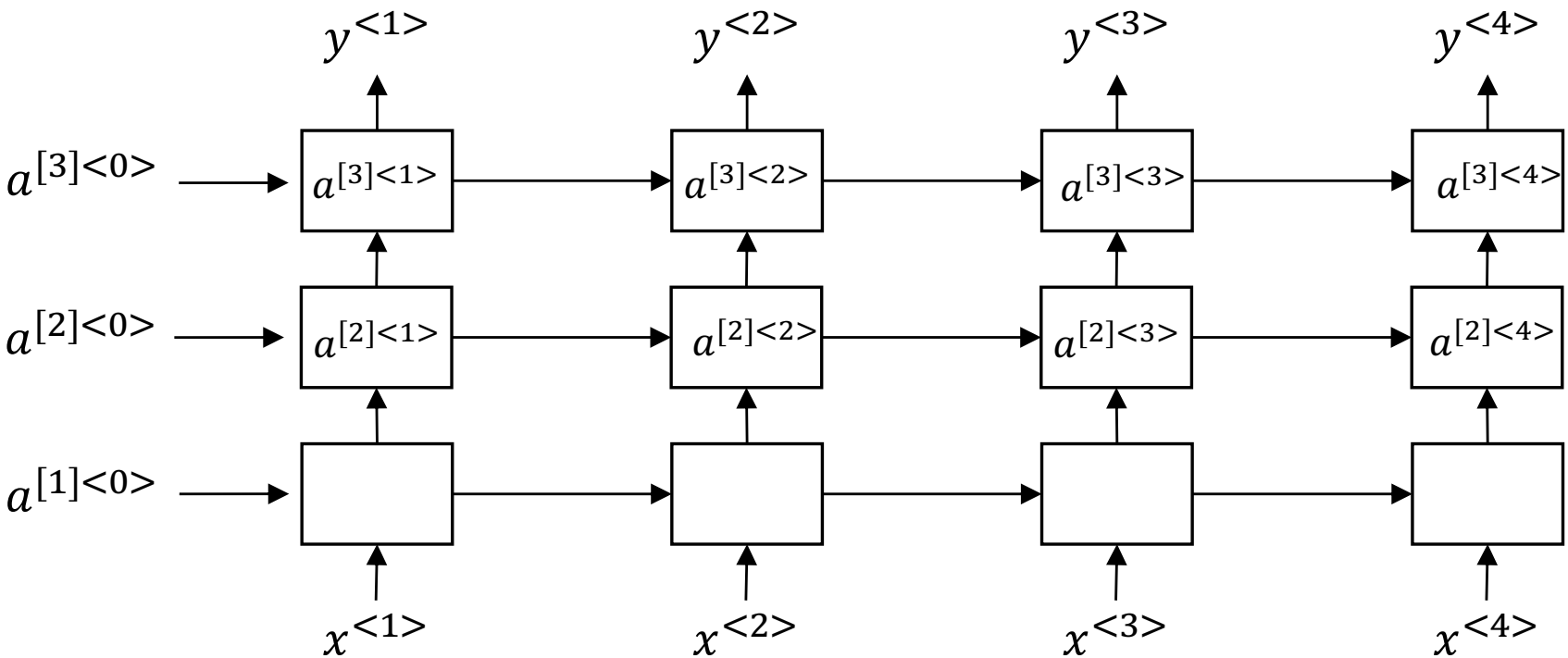
Andrew Ng

Example RNN- IMDB

```
class RNNClassifier (nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers=1):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.rnn = nn.RNN(input_size=embed_dim, hidden_size=hidden_dim,
                          num_layers=num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):    # x shape: (batch_size, seq_len)
        embedded = self.embedding(x)
        output, hidden = self.rnn(embedded)
        last_hidden = hidden[-1]
        out = self.fc(last_hidden)
        return out.squeeze(1)
```

imdb_torch_full.py

RNNs with multiple layers



Andrew Ng

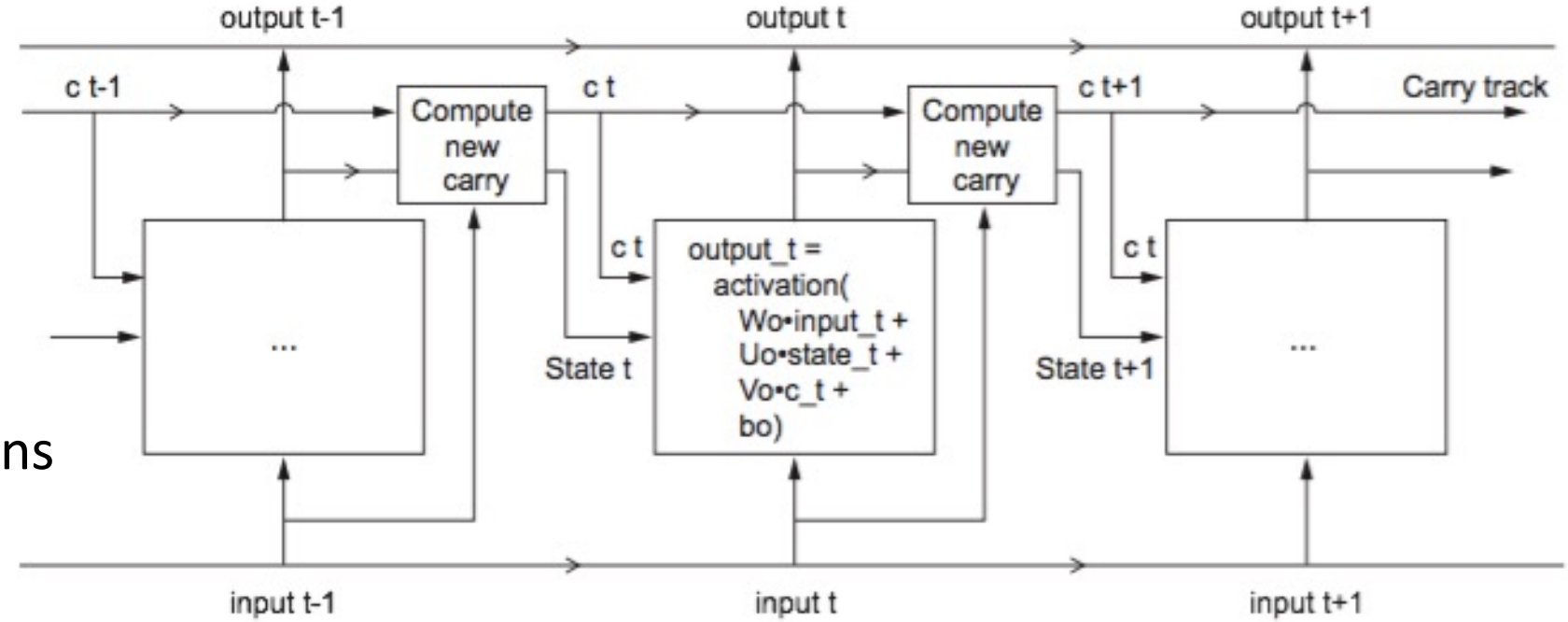
LSTMs

LSTM – **Long Short-Term Memory** – are recurrent networks that are characterized by maintaining "memory"

They have been introduced to solve the so-called problems of ”vanishing gradients”

c_t – *carry*

Calculated from 3 transformations

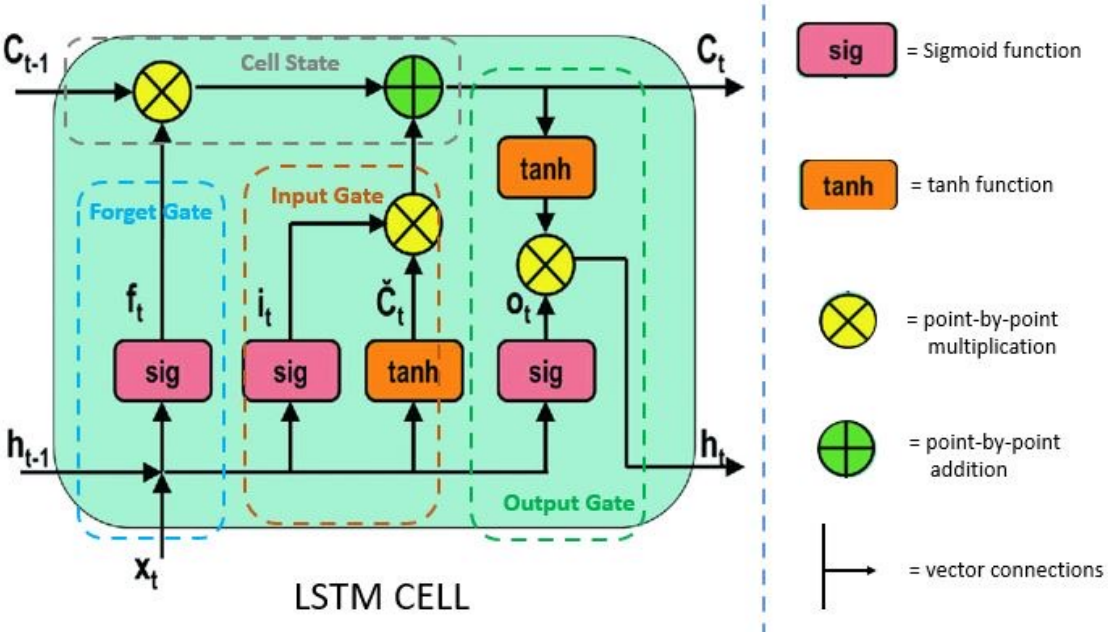


LSTMs - pseudocode

```
output_t = activation(dot(state_t, Uo) + dot(input_t, Wo) + dot(c_t, Vo) + bo)
i_t = activation(dot(state_t, Ui) + dot(input_t, Wi) + bi)
f_t = activation(dot(state_t, Uf) + dot(input_t, Wf) + bf)
k_t = activation(dot(state_t, Uk) + dot(input_t, Wk) + bk)
```

```
c_t+1 = i_t * k_t + c_t * f_t
```

Names of the gates (forget, input, …) are merely indicative of their perceived “role”



Example LSTM - IMDB

```
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_layers=1, dropout=0.0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM( input_size=embed_dim,
            hidden_size=hidden_dim, num_layers=num_layers,
            batch_first=True, dropout=dropout if num_layers > 1 else 0.0)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x): # x shape: (batch_size, seq_len)
        embedded = self.embedding(x)    # (B, L, E)
        output, (hidden, cell) = self.lstm(embedded)
        last_hidden = hidden[-1]        # Take hidden state from last LSTM layer
        out = self.fc(last_hidden)
        return out.squeeze(1)
```

imdb_torch_full.py

Recurrent networks: supplementary notes

LSTM layers can be replaced by **GRU layers** with similar functionality (simpler – fewer weights to train)

GRU or LSTM layers can be **stacked** creating models with higher capacity (parameter num_layers in Pytorch)

You can consider using **Dropout** in these layers to address overfitting (check the examples in the python code)

Bidirectional recurrent networks may be used in cases where both the natural order of the sequences and their reverse may make sense (flag defined in the code)

Multiple classes

In many cases, we are interested in classifying texts in a number of classes larger than 2

One example will be provided by the practical exercise in this class !!

Main changes:

- Dense layer in the output – number of neurons
- Loss function

Evaluating DL models

Training

Used to train models

Validation
(*dev set*)

Used to optimize models/ hiper-parameters

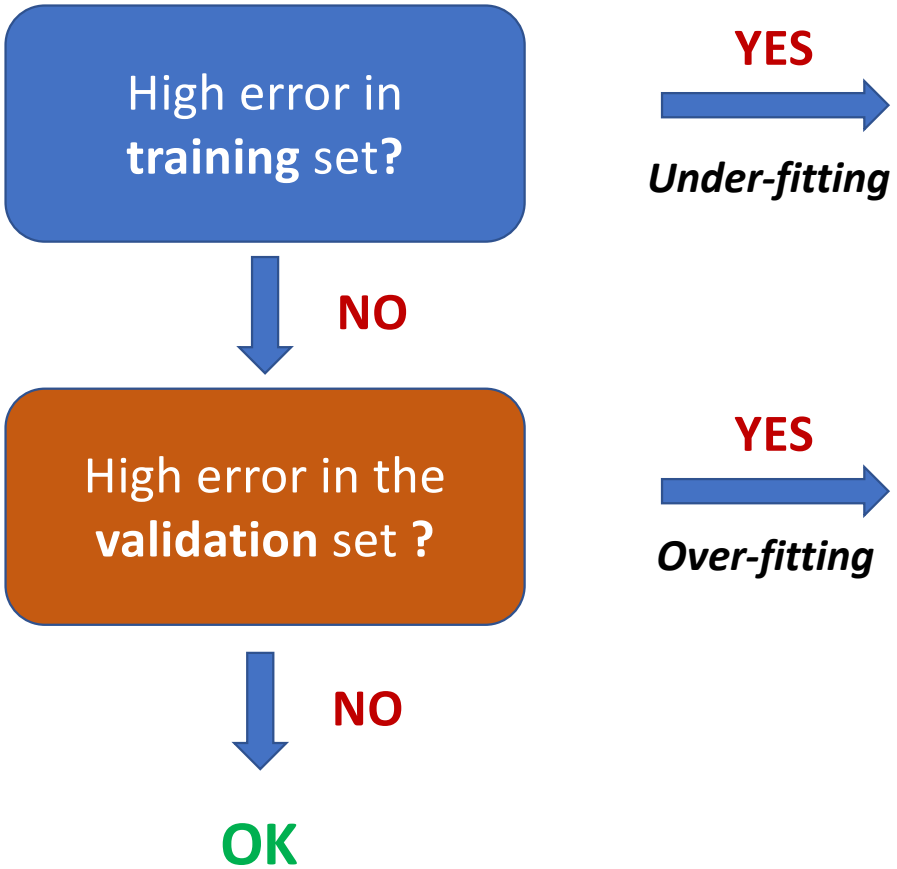
Test

Used to estimate error in the final model and comparing models

Over and underfitting

Error in **training**	↑	↓	↓
Bias	Under-fitting		
Error in **validation**	↑	↑	↓
Variance		Over-fitting	OK

Workflow



Larger/more complex network
More training time
Better training algorithms
Algorithm parameters
Other architectures ??

More data
Regularization, dropout, …
Less training time: early stopping
Other architectures ??

Hyperparameter optimization in DNNs

As with all supervised learning approaches, DL models should be the subject of a hyper-parameter optimization process to ensure their best performance. In this case, the most relevant hyper-parameters include:

- Number of layers
- Number of neurons in each layer
- Activation functions to use
- Training algorithm and its parameters (e.g. learning rate)
- Use of dropout and its rate
- Use of regularization and associated parameters
- Using early stopping or not; related parameters

and many others when considering RNNs, LSTMs, … and other topologies …

Hyperparameter optimization

hyperparopt.py

Working for DNNs – can be extended to other models

```
class FFNN (nn.Module):
    def __init__(self, nfeatures, nclasses = 2, topology = [50], algorithm = "adam",
                 dropout = 0.0, learning_rate = 0.001, l2 = 0.0):
```

Sets models (DNNs) for classification (binary/ multiclass), receiving topology, learning algorithm, learning rate, dropout rate, and L2

```
    def train_model(model, train_loader, val_loader, epochs = 20, …):
        …
```

Trains model (DNN); receives training and validation data, epochs

```
def dnn_optimization(opt_params, train_loader, val_loader, iterations = 10, epochs = 5, ..):
…
```

Hyperparameter optimization (opt_params gives the list of options for each parameter)
Algorithm used: randomized search